HBC RADIO

ATAK Plugin — Interface Control Document

Ham Binary CoT over Distance-Vector Mesh Radio

Version 1.8 | KE8TQB | github.com/ke8tqb/HBC-ATAK-Plugin

Table of Contents

1. Overview
2. System Architecture
    - 2.1 C2 Bridge — One-Way RF → Network Data Diode
3. Physical Layer (Audio Modems)
    - 3.1 AFSK1200 / AX.25
    - 3.2 OFDM COFDMTV (rattlegram)
    - 3.3 Mercury HF (FreeDV DATAC4)
    - 3.4 PSK31 (BPSK31 with text armor)
    - 3.5 Channel Access — CSMA (contention)
    - 3.6 Channel Access — Ring MAC (deterministic rotation)
4. Mesh Network Layer
    - 4.1 Common Packet Header
    - 4.2 Callsign Encoding (ITA2, 5 bytes)
    - 4.3 Packet Type 0x01 — Announce
    - 4.4 Packet Type 0x02 — Broadcast
    - 4.5 Packet Type 0x03 — Direct Message
    - 4.6 Packet Type 0x04 — Acknowledgment
    - 4.7 Route Discovery and Maintenance
    - 4.8 Loop Prevention
    - 4.9 Reliability and Retries
5. Application Layer (HBC Protocol v1.6)
    - 5.1 HBC Header
    - 5.2 Message Modes and Sizes
    - 5.3 Deterministic UIDs
    - 5.4 Direct-Message Tags and Receipts
    - 5.5 Contact Publication
6. User Interface Reference (Screenshots)
    - 6.1 Screen-by-Screen Figures
    - 6.2 Every Control and What It Does
    - 6.3 Toolbar Icons

6.4 Reading the Activity Log
6.5 Reading the Decodes Log
7. Timing Parameters and Constants
8. Compatibility and Constraints
9. Glossary
10. References

Appendix A — GitHub Repository File Structure

Revision History

ICD rev	Date	Summary of changes
1.8	Oct 2026	Added the PSK31 modem (section 3.4), its airtime-scaled channel-access values throughout, and the new modem-selector figure. Document made fully self-contained: feature history is consolidated here instead of inline version notes.
1.7	Oct 2026	Added the TX output / hardware-PTT selector (Digirig RTS and GPIO keying), USB hot-plug receive recovery and USB TX-level guidance; refreshed the full screenshot set.
1.6	Oct 2026	Added the C2 Bridge one-way RF → network data diode (section 2.1) and its Push to RF workflow.
1.0–1.5	Sep–Oct 2026	Initial document: three-layer architecture, the first three modems, the mesh network layer, the Ring MAC, ARQ refinements and the UI reference.

1. Overview

HBC Radio is a plugin (an add-on) for ATAK, the Android Team Awareness Kit mapping app. It lets ATAK devices share map information — positions, markers, chat messages, and alerts — using ordinary handheld ham radios instead of the internet or a cell network. The phone plays specially coded sound into the radio; another phone hears that sound on its radio and turns it back into map data.

The system is built in three layers, like a stack of responsibilities. Each layer is specified in its own section of this document:

- **Physical layer (Section 3)** — the “voice” of the system: one of four selectable audio modems that turn bytes into sound and back.
- **Mesh network layer (Section 4)** — the “postal service”: it figures out who can hear whom and can pass a message through a middle station to reach someone out of range. Based on a published distance-vector routing design (see References).
- **Application layer (Section 5)** — the “language”: the HBC Protocol, which squeezes a map event into a few dozen bytes so it can be sent quickly over a slow radio link.

> **In plain terms:** HBC (the language) writes a very short letter, the mesh (the postal service) addresses it and picks the route, and the modem (the voice) reads it out loud over the radio.

> **NOTE:** Everything is transmitted unencrypted (in the clear), as required by amateur-radio rules (FCC Part 97). The operator’s callsign is inside every transmission, which satisfies station-identification requirements.

2. System Architecture

The path one message takes from your screen to the other station’s screen:

```
ATAK map event → HBC encoder (shrinks it to bytes) → MeshRouter (adds addressing) → modem (turns bytes into audio) → radio → ...air... → radio → modem → MeshRouter → HBC decoder → ATAK map
```

On receive, the mesh layer decides what to do with each frame: deliver it to this device, forward it toward its destination, rebroadcast it (announces), or ignore it.

Layer	Code that implements it	How things are addressed
Application	HbcEncoder / HbcDecoder	Callsign inside the HBC header; predictable UIDs (section 5.3)

Network	MeshRouter	Origin / Transmitter / Destination / Next-hop callsigns
Physical	AudioModem, OfdmModem, MercuryModem, Psk31Modem	None — just a byte pipe (AFSK adds a fixed AX.25 wrapper for station ID)

2.1 C2 Bridge — One-Way RF → Network Data Diode

A station that hears the radio mesh AND sits on a normal ATAK network at the same time (WiFi mesh SA or a TAK server connection) can act as a **C2 Bridge**: a command-post gateway that pushes the radio picture onto the network without ever letting network-volume traffic flood the slow radio channel. Enabled by the **C2 Bridge** checkbox (Options tab); the header shows · **BRIDGE** while active.

- **Radio → network (automatic).** Every event decoded from the radio — positions, markers, 911 alerts, shapes, CASEVAC, broadcast/room chat — is re-published onto the ATAK network outputs so command-post users see it. Mode 0 receipts (radio ARQ plumbing) and direct messages addressed to the bridge operator (private) are not forwarded. Forwarded position reports are re-rendered *without* the chat endpoint: network users see the radio stations on the map but cannot open a chat to a station that could never answer through a one-way bridge.
- **Network → radio (manual only).** The bridge phone still displays everything the network sends it, but NOTHING network-originated is automatically transmitted over radio. The bridge's own traffic (its PLI, its operator's markers and chats, receipts for radio DMs) transmits normally; everything else is blocked and kept for 10 minutes in the **Push to RF** list, where the operator hand-picks the individual items worth airtime — the human is the rate limiter. Radio-origin events are tagged when forwarded so they can never echo back onto the air.

> **NOTE:** The bridge is pure local behavior on the bridge phone — nothing on the air changes, remote stations need no update, and the one-way design also keeps unlicensed network users' traffic off the amateur bands (FCC Part 97): the only transmissions are initiated by the control operator's own station, automatically for its own data or deliberately via Push to RF.

3. Physical Layer (Audio Modems)

A modem converts bytes into sound and sound back into bytes. HBC Radio offers four, each suited to different radios and conditions. All four carry the exact same mesh frame, but they are not compatible with each other — **every station on a channel must select the same modem** (Figure 3).

Parameter	AFSK1200	OFDM COFDMTV	Mercury HF	PSK31

Waveform	Bell 202 AFSK, 1200 baud	QPSK OFDM (rattlegram “short”), 1600 Hz BW	FreeDV DATAC4 raw-data OFDM (codec2)	BPSK 31.25 Bd (PSK31 Varicode), ~31 Hz BW
Sample rate	48000 Hz	48000 Hz	8000 Hz	8000 Hz
Payload capacity per burst	Practical AX.25 limits (>200 B)	169 B	52 B (54 B frame minus framing)	180 B (one frame per burst)
Error protection	HDLC CRC-16 (detect only)	Polar code + CRC-32	LDPC + CRC-16	CRC-16/X.25 on the armored frame (detect only)
Best for	VHF/UHF FM; compatible with classic packet radio	VHF/UHF FM; survives phone audio processing	HF SSB; works at very weak signal levels	Any voice channel incl. HF SSB; extreme weak signal; very slow

3.1 AFSK1200 / AX.25

The classic 1980s packet-radio sound (two alternating tones, 1200 bits per second). Mesh frames ride inside a single AX.25 “UI” frame. The AX.25 addresses are only wrapping paper — they are not used for routing: destination is always `HBC`, source is the operator’s ham callsign (which doubles as the legal station ID), and no digipeater path is used. Each transmission is rendered in advance as: lead-in silence (150 ms) → optional VOX leader tone → HDLC flag preamble (“TX Dwell,” default 500 ms) → the frame → tail silence (250 ms).

3.2 OFDM COFDMTV (rattlegram)

A modern, robust waveform that packs data onto many small carriers at once. Framing inside the 170-byte block: byte 0 = mesh frame length (1–169), bytes 1..n = the mesh frame, zero-padded. The operator callsign also rides in the waveform’s own metadata header. Audio carrier is 1500 Hz; the VOX leader is made of noise symbols (~180 ms each).

3.3 Mercury HF (FreeDV DATAC4)

Designed for long-range HF radio, where signals are weak and distorted. The burst layout matches the Mercury HF modem exactly: preamble → one DATAC4 data frame with a CRC-16 checksum in its last two bytes → postamble, with 100 ms of silence on each end. Framing inside the 54 usable bytes: byte 0 = length, byte 1 = the letter `'H'` (a marker that says “this is HBC traffic”), bytes 2..n = the mesh frame, zero-padded.

> **NOTE:** This speaks Mercury’s *waveform*, not Mercury’s higher-level ARQ data-link protocol. A desktop Mercury installation can demodulate these bursts but will not pass them to its TCP data

> port.

3.4 PSK31 (BPSK31 with text armor)

The narrowest and slowest modem: 31.25-baud binary phase-shift keying with the ITU-R M.2034 Varicode alphabet — the same waveform human operators use for keyboard-to-keyboard PSK31 contacts. Audio carrier 1500 Hz, sample rate 8000 Hz, occupied bandwidth about 31 Hz: it drops into any SSB voice channel alongside human PSK31 signals and keeps decoding at signal-to-noise ratios where even the OFDM modes fail. The price is airtime — roughly 0.4 s per byte plus ~4 s of preamble/postamble: a position report is ~16 s of continuous audio, a full extended marker ~30 s, and the largest accepted frame (180 bytes) ~77 s.

Varicode itself carries no error detection, so binary mesh frames ride inside a text *armor*: ~ + Base64(mesh frame + CRC-16/X.25) + ~. The receiver ignores anything on frequency that is not a well-formed armored frame (human PSK31 chatter, noise), and the CRC rejects corrupted frames outright — a damaged frame is dropped, never delivered wrong. The VOX leader is a steady carrier tone (an idle PSK31 carrier): it trips VOX radios and decodes as nothing.

> **Channel access at PSK31 speeds:** every channel-access time constant scales with this modem's airtime. The Ring MAC sizes its turn deadline from the worst-case frame airtime (~94 s) and releases exactly **one frame per turn**; CSMA waits up to 90 s — longer than the longest possible neighbor burst — before its give-up override; the carrier-sense re-baseline window is likewise 90 s, so a neighbor's long continuous carrier is never mistaken for a stuck-open squelch; and the position-report floor is 25 s per roster station (section 3.6). Expect a deliberately slow reporting cadence — PSK31 trades speed for reach.

> **Operating guidance:** with three or more stations, run the Ring MAC (section 3.6). At these burst lengths a single collision wastes 30 s or more of channel time; the ring's collision probability is zero by construction, while CSMA's grows with burst duration.

3.5 Channel Access — CSMA (contention)

Only one station can talk on a radio channel at a time; two at once garble both (a "collision"). In CSMA mode (the default), every modem *listens first*: incoming audio energy is compared against an automatically learned background-noise level, and the channel counts as busy when audio is 4× louder than that background (held for 400 ms past the last loud moment so brief gaps inside someone's transmission don't look like free air). When the channel goes quiet, the station still waits a random 150–550 ms — so two stations that were both waiting don't key up at the same instant. After 8 seconds of continuous busy (15 s on Mercury, 90 s on PSK31) the frame is sent anyway so traffic can never be blocked forever. AFSK also uses its demodulator's data-carrier-detect as a second "someone is talking" signal.

> **When to use which:** CSMA is fine for 2–5 stations. It cannot see *hidden terminals* (two stations that cannot hear each other but both reach a third), and with VOX radios every frame pays the radio's full key-up/hang cycle — so with many stations, collisions and dead air grow. For nets up to ~20 stations — or PSK31 nets of 3 or more (section 3.4) — select the **Ring MAC** (section 3.6) on every station instead.

3.6 Channel Access — Ring MAC (deterministic rotation)

Instead of competing for the channel, stations take **turns in a fixed, known order**: every station sorts the callsigns it knows about (its own plus the mesh address book of section 4.7, which fills itself just by listening) and rotates through that list. The list IS the schedule — there is no coordinator station, no "token" packet, and **no change to anything transmitted on the air**: the MAC only decides *when* a station may key up, never what the bytes look like. A CSMA station in a Ring net is simply heard as carrier and deferred to.

A station transmits only during its own turn — and then sends **everything it has queued as one continuous keying** (one batch: single lead-in, frames back-to-back, single tail), so the radio's VOX hang is paid once per turn instead of once per frame. Every station advances the turn pointer on its own, using whichever of these happens first:

Rule	Trigger	Purpose
1. Early release	The owner's carrier was heard and the channel has then stayed clear for **Guard** ms	Short messages finish early; Guard absorbs the radio's VOX hang so the tail is never mistaken for free air
2. Silent skip	No carrier within **Skip** ms of the turn starting	Idle stations cost ~1 s, not a whole turn
3. Deadline	**Max turn + Guard** ms since the turn started	Keeps the ring alive past hidden terminals and dead stations — even a station that hears nothing advances

Two additional behaviors keep every station's pointer agreeing: decoding any mesh frame **re-aligns the pointer to that frame's Transmitter callsign** (hard evidence beats timers), and a station never keys up while the channel is busy (interlock), even on its own turn. A joining station listens for one full cycle (at most 30 s) before taking its first turn; roster changes apply only when the rotation wraps. A station that stays silent for 3 consecutive turns is noted as dormant (it keeps costing only the Skip time until the mesh route TTL removes it). 911 alerts are the one exception to turn order: they may transmit in the next idle gap between turns after a short random delay.

Setting (Audio Setup tab)	Default	What it covers

Guard ms	1500	The radio's VOX hang after audio stops, plus margin
Skip ms	1200	VOX attack (~250 ms) + time to detect the carrier before declaring a station silent
Max turn ms	0 = automatic per modem (AFSK 6000, OFDM ~4400, Mercury ~6000, PSK31 ~94000 — sized from the worst-case frame airtime)	The longest allowed batch; also the deadline timer

> **Radio profile — Baofeng UV-5R and Yaesu FT-65 (reference radios):** neither radio's VOX hang is adjustable — the FT-65's VOX menu is ON/OFF only (Set Mode item 35, used with the SSM-512B earpiece mic) and the UV-5R exposes only VOX *sensitivity* 1–10 — and both hold the transmitter keyed roughly 1.0–1.5 s after audio stops. The 1500 ms Guard default covers that worst case. Their VOX attack also clips the first ~150–250 ms of audio, so set **VOX Lead = 250 ms** when driving these radios. Both radios' time-out timers (≥60 s) are far above the longest turn on the fast modems — no interaction (on PSK31, whose bursts can exceed 60 s, disable or maximize the radio's time-out timer). With a Digirig-class USB interface, select **TX output = USB + RTS PTT** instead: the plugin keys the radio electrically, VOX is out of the picture entirely, and Guard 300–500 ms / VOX Lead 0 apply.

> **Calibrating Guard:** run a two-station session, key one radio with a short test transmission, and read the session log's receive timing (the gap between the end of decoded audio and the next burst, or the AFSK RX burst lines) to measure how long the carrier outlives the audio. Set Guard = the worst measured hang among all radios in the net + 150 ms. Every station must use the same Guard, Skip and modem.

Capacity (20 stations, 5 of them with traffic in a given cycle, defaults): AFSK ≈ 32 s per full rotation, OFDM ≈ 35 s, Mercury ≈ 54 s. PSK31 is in a different class — a single position report is ~16 s of audio, so even a 5-talker cycle runs several minutes (section 3.4). So that position reports never dominate the rotation, Ring mode automatically floors the PLI interval at `roster × 8 s` (`× 13 s` on Mercury, `× 25 s` on PSK31), even if the user's "PLI min s" is set lower — that keeps PLIs at roughly a quarter of each station's turn capacity (about 60% on PSK31), leaving the rest for chat, ACKs and relays.

> **ARQ interaction:** the mesh layer's delivery retries (section 4.9) are aware of the Ring MAC — a retry timer tuned for contention access would otherwise fire while the original Direct was still waiting for its turn, duplicating traffic and flooding the queue: (1) a pending Direct's retry clock only starts once its latest copy actually *airs*; (2) under Ring the retry interval scales to at least 1.25× the measured rotation time (an ACK cannot physically return sooner than the recipient's own turn); (3) chat delivered/read receipts travel as unacknowledged Directs; (4) a queued position report is replaced in place when a fresher one arrives; and (5) a persistent backlog is called out in

> the Activity Log (`Ring: TX queue…`). Raising **Max turn** (identically on every station) also raises how many frames fit in one batch, so operators can trade longer turns for faster queue drain.

> **Design notes (why not the alternatives):** REQUEST/GRANT schemes, RTS/CTS and explicit token packets all cost full modem bursts per exchange — on OFDM every burst is ~1.84 s, on Mercury ~5.6 s, and on PSK31 tens of seconds regardless of payload size, so control traffic is as expensive as data. A coordinator is also a single point of failure. Slotted TDMA wastes the slot remainder on short messages. The ring keeps the determinism of TDMA/token designs with none of their airtime overhead.

4. Mesh Network Layer

Implements “Adaptation of Uncoordinated Distance-Vector Routing for Unencrypted Amateur Radio Networks” (see References), with one approved deviation: packet payloads carry compact HBC binary rather than ITA2 text.

> **In plain terms:** every station keeps a simple address book (“to reach DAVE, hand the message to BOB”). The address book is built just by listening, and messages for far-away stations get passed station-to-station like a relay race.

4.1 Common Packet Header (13 bytes)

Every mesh packet starts with the same 13 bytes:

Byte offset	Length	Field	Description
0	1	Packet Type	0x01 Announce, 0x02 Broadcast, 0x03 Direct, 0x04 ACK
1	5	Origin Callsign	Who originally created the packet (bit-packed ITA2, section 4.2)
6	5	Transmitter Callsign	Who is transmitting this particular hop; each relay overwrites it with its own callsign
11	2	Sequence ID	A 16-bit counter value (big-endian) used to recognize duplicates (section 4.8)

4.2 Callsign Encoding (ITA2, 5 bytes)

Callsigns are stored in an old teletype alphabet called ITA2, which uses only 5 bits per character instead of 8 — so 8 characters fit in exactly 5 bytes (40 bits). Two special “shift” codes switch between letters (LTRS = `11111`) and figures/digits (FIGS = `11011`); unused positions are filled with NULL (`00000`). A leading LTRS is written before the first letter; if a callsign would need 9 codes (for example `KE8TQB`), the leading LTRS is dropped — decoders start in letters mode anyway, so both forms read back identically.

Example	Code sequence	Packed bytes (hex)
K1ABC	LTRS K FIGS 1 LTRS A B C	`FF B7 DF E2 6E`
N3DEF	LTRS N FIGS 3 LTRS D E F	`F9 B7 0F CA 16`
W2XYZ	LTRS W FIGS 2 LTRS X Y Z	`FE 77 9F DE B1`

> **NOTE:** The source paper's K1ABC example lists byte 03 as `BF`, but the paper's own bit-by-bit math works out to `DF`; this implementation follows the math. All three vectors above are locked in by the automated test suite (`codec-test/MeshRouterTest.java`).

4.3 Packet Type 0x01 — Announce (14 bytes)

A tiny "I exist and here's how far away I am" packet that builds everyone's address book.

Byte offset	Length	Field	Description
0–12	13	Common Header	Type = 0x01
13	1	Hop Count	How many relays the announce has passed through; the origin sets 0, each relay adds 1

4.4 Packet Type 0x02 — Broadcast (13 + n bytes)

A message to everyone in earshot. Not relayed, not acknowledged.

Byte offset	Length	Field	Description
0–12	13	Common Header	Type = 0x02
13+	variable	Payload	One HBC message (section 5)

4.5 Packet Type 0x03 — Direct Message (23 + n bytes)

A message for one specific station, passed hop-by-hop.

Byte offset	Length	Field	Description
0–12	13	Common Header	Type = 0x03
13	5	Destination Callsign	The final recipient
18	5	Next Hop Callsign	The neighbor being asked to carry this hop; each relay rewrites it
23+	variable	Payload	One HBC message

A station that sees its own callsign in Next Hop looks the Destination up in its address book, writes in the next station's callsign, and re-transmits. The final destination delivers the payload and answers with an ACK (section 4.6). If nobody knows a route, a relay drops the packet; the original sender instead falls back to sending it as a Broadcast.

4.6 Packet Type 0x04 — Acknowledgment (20 bytes)

The “got it!” receipt for a Direct message, sent back to the original sender.

Byte offset	Length	Field	Description
0–12	13	Common Header	Type = 0x04. Origin = the station acknowledging.
13	5	Destination Callsign	The Origin of the Direct being acknowledged
18	2	Acked Sequence ID	The Sequence ID of the Direct being confirmed

Stations along the way forward ACKs toward their destination when they know a route; the duplicate cache (section 4.8) keeps that from snowballing.

4.7 Route Discovery and Maintenance

- **Announce propagation** — when a station hears a new Announce, it writes the entry “Origin is reachable via whoever just transmitted this” and re-broadcasts the announce with hop count + 1. Shorter routes (fewer hops) replace longer ones; ties go to whichever route was heard first.
- **Passive learning** — hearing *any* valid frame is proof of a path, so the transmitting station is recorded as a direct neighbor, and (for non-announce frames) the originating station is recorded as reachable through that transmitter. Hearing a single position report is enough to be able to send to that station.
- **Announce-on-traffic** — a station’s own transmissions do the announcing for it; an explicit Announce packet only goes out after 10 quiet minutes, as a keepalive.
- **Expiration** — address-book entries are erased 30 minutes after the last frame that refreshed them (stations move and links change); a cleanup pass runs every 60 s.

4.8 Loop Prevention

Without protection, a rebroadcast message could bounce between stations forever. Each receiver therefore remembers `[Origin + Sequence ID + Type]` combinations for 5 minutes; a packet it has already seen is silently ignored. One deliberate exception: a duplicate Direct addressed *to this station* gets a fresh ACK (but is not delivered twice) — that covers the case where the first ACK was lost on the air.

4.9 Reliability and Retries

Directs work “fire and wait”: the originator keeps the message until the destination’s ACK arrives, re-sending it **with the same Sequence ID** up to 3 times, then declaring it failed and logging it. Broadcasts are never acknowledged or retried. Two refinements let this ARQ coexist with a queueing MAC (the Ring):

- **The retry clock starts when the frame airs, not when it is queued.** Under the Ring MAC a frame can wait many seconds for this station's turn; retrying while the original is still queued only duplicates it in the same queue. A pending whose copies never reach the air at all is failed after 3 minutes (stuck-queue safety).
- **Retry pacing follows the channel-access mode.** CSMA: 5 s + 0–2 s random jitter (the original behavior). Ring: at least 1.25× the measured rotation time (minimum 8 s, capped at 60 s) — an ACK cannot return before the recipient's own turn comes around, so earlier retries are pure overhead.

Chat delivered/read receipts (section 5.4) are sent as **unacknowledged Directs**: identical on the wire, but with no retries and no failure verdict — a lost checkmark does not justify retry traffic. The recipient still answers with a mesh ACK, which the origin simply ignores.

5. Application Layer (HBC Protocol v1.6)

The payload inside every Broadcast/Direct is one HBC message: a map event squeezed into as few bytes as possible by packing values bit-by-bit instead of sending bulky XML text. (For scale: a normal ATAK position report is roughly 1–2 *thousand* bytes of XML; the HBC version is about 15 bytes.) Full field-level definitions and worked byte examples live in the HBC-Protocol repository (see References); this section summarizes the interface.

5.1 HBC Header

Field	Size	Description
Callsign	variable (ITA2, CR-terminated)	Sending operator/device callsign, max 8 characters
Version	3 bits	`000` = version 1
Mode	3 bits	mode−1 for Modes 1–6; `111` = Mode 0 Ack

> **NOTE:** HBC uses its own ITA2 table (bit-reversed relative to the mesh header's table) and marks the end of text with a CR code instead of padding to a fixed size. The two layers' callsign encodings are cousins, not twins — they are not interchangeable.

5.2 Message Modes and Sizes

Mode	CoT type(s)	What it carries	Typical size
0 Ack	b-t-f-d / b-t-f-r	Chat delivered/read receipt: recipient callsign, kind (2 bits), 16-bit tag	~12 B
1 PLI / Spot	a-f-G, a-h-G, a-n-G, a-u-G	Position report: spot flag, affiliation (2 bits), name (7 chars), latitude/longitude (21+22 bits)	~15 B
2 Alert	b-a-o-tbl / b-a-o-can	911 emergency: active flag, alert + originator names, position	~18 B
3 GeoChat	b-t-f	Text message: destination kind (All / Room / DM), room or recipient, DM 16-bit tag, ITA2 text	DM "helo landing zone" = 23 B
4 Shape	u-d-c-c / u-d-r / u-d-f	Drawing: circle (center + radius) or polygon/polyline (≤15 points)	varies
5 CASEVAC	b-r-f-h-c	Medical evacuation request (9-line summary fields)	~20 B
6 Extended Marker	any placed marker	Full CoT type token, icon reference (2525C / spot-map / custom), color tint	varies

5.3 Deterministic UIDs

ATAK identifies every map object by a UID (unique ID string). HBC does not transmit UIDs — both ends compute the same one from the callsign, so a station's position marker updates in place instead of piling up duplicates:

Object	UID rule
Position (PLI) marker	`HBC-{CALLSIGN}`
911 alert	`HBC-{CALLSIGN}-911`
DM conversation	Recipient's UID; the receiving plugin swaps in its real ATAK device UID so the message lands in the right chat window
Spots / shapes / markers	Random UUID each time one is received

5.4 Direct-Message Tags and Receipts

Every chat DM carries a 16-bit tag — a CRC-16/CCITT-FALSE checksum (polynomial 0x1021, initial value 0xFFFF) of ATAK's internal `messageId`. When the recipient's ATAK automatically generates its delivered/read receipts, the plugin converts them into tiny Mode 0 Acks that echo the tag back over the air; the sender matches the tag to the original messageId and injects the receipt, so ATAK's familiar delivered/read checkmarks work end-to-end over radio. These application-level receipts are separate from (and in addition to) the mesh-layer ACKs of section 4.6. They are carried as unacknowledged Directs (section 4.9) — best-effort by design.

5.5 Contact Publication

ATAK only lists a station in its chat/contact pickers when that station's position report includes a network "endpoint." Received PLIs are therefore reconstructed with `<contact callsign="..." endpoint="*:-1:stcp"/>` and a `<__group>` team entry, so hearing a single position report makes the station selectable for chat and file/point sends. Outgoing traffic to that contact is intercepted by the plugin and transmitted over the mesh.

6. User Interface Reference (Screenshots)

All configuration lives in one pane, opened from the HBC Radio entry in ATAK's Tools menu (Figure 1). This section shows each screen (6.1), explains every control (6.2) and toolbar icon (6.3), and teaches how to read the two logs (6.4, 6.5). Red boxes and numbered markers in the figures point at the controls each caption describes.

6.1 Screen-by-Screen Figures

All figures show the plugin pane: three tabs (**Audio Setup**, **Options**, **Decodes**) under a slim always-visible header carrying the colored status line, the build stamp and the Start/Stop button.



*Figure 1 — Opening the plugin: ATAK menu (☰) → Tools → **HBC Radio** (boxed). The Tools grid tints every icon white — the tower glyph is the plugin's single entry point (section 6.3).*

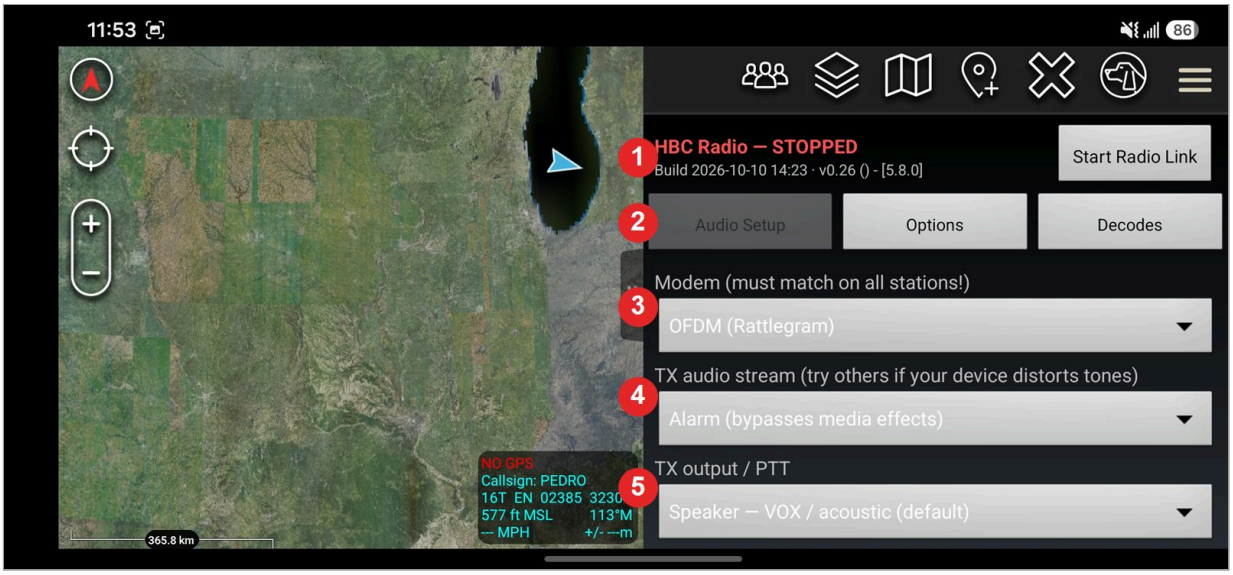


Figure 2 — Audio Setup tab, top. ① status line (red STOPPED / green RUNNING) with the build stamp beneath it; ② tab bar with the always-visible Start/Stop button at its right; ③ Modem selector (must match on all stations); ④ TX audio stream; ⑤ TX output / PTT (section 6.2).

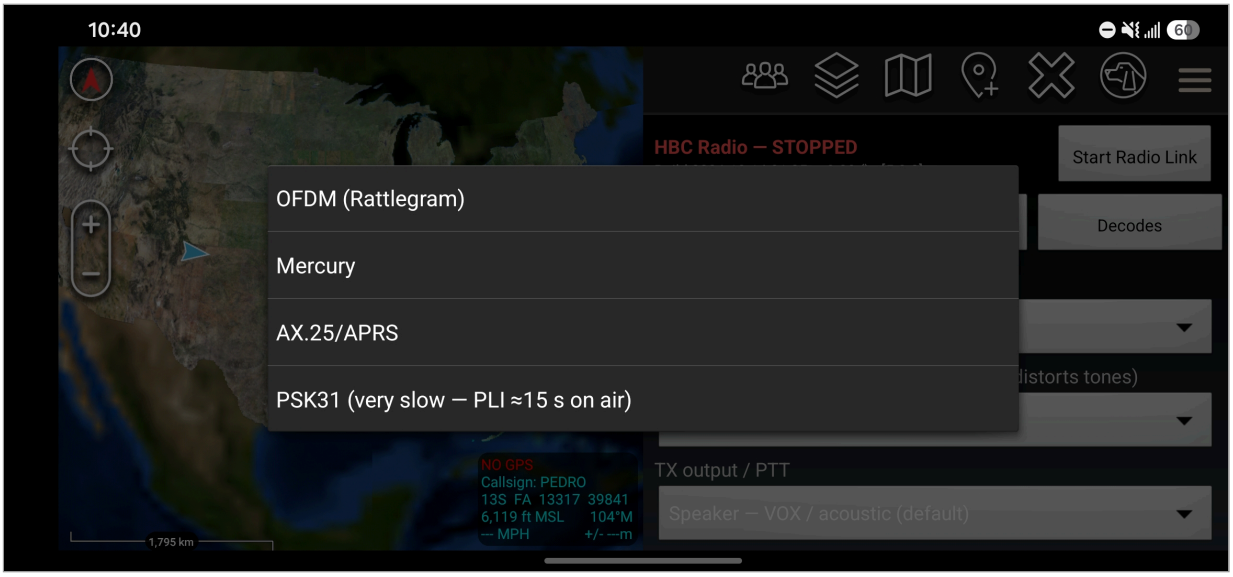


Figure 3 — The modem selector expanded (section 3): OFDM (Rattlegram, the default), Mercury, AX.25/APRS, and PSK31 — whose entry carries its speed warning right in the label (very slow — PLI ≈15 s on air, section 3.4). All stations on a channel must choose the same modem.

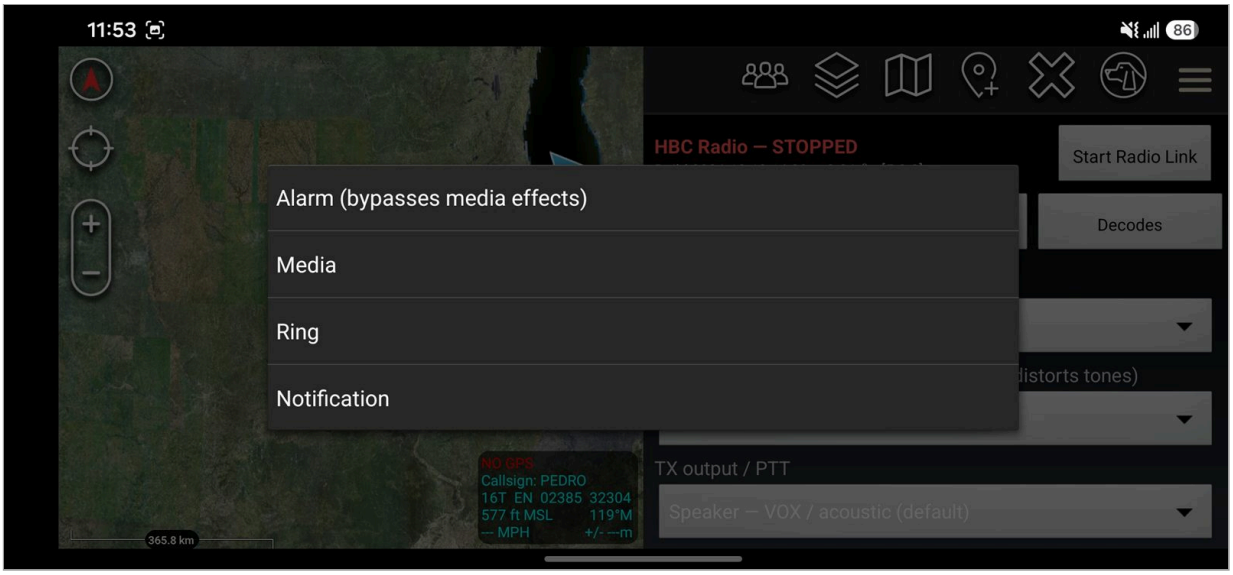


Figure 4 — TX audio stream expanded: Alarm (default — bypasses the phone media effects that distort modem tones), Media, Ring, Notification.

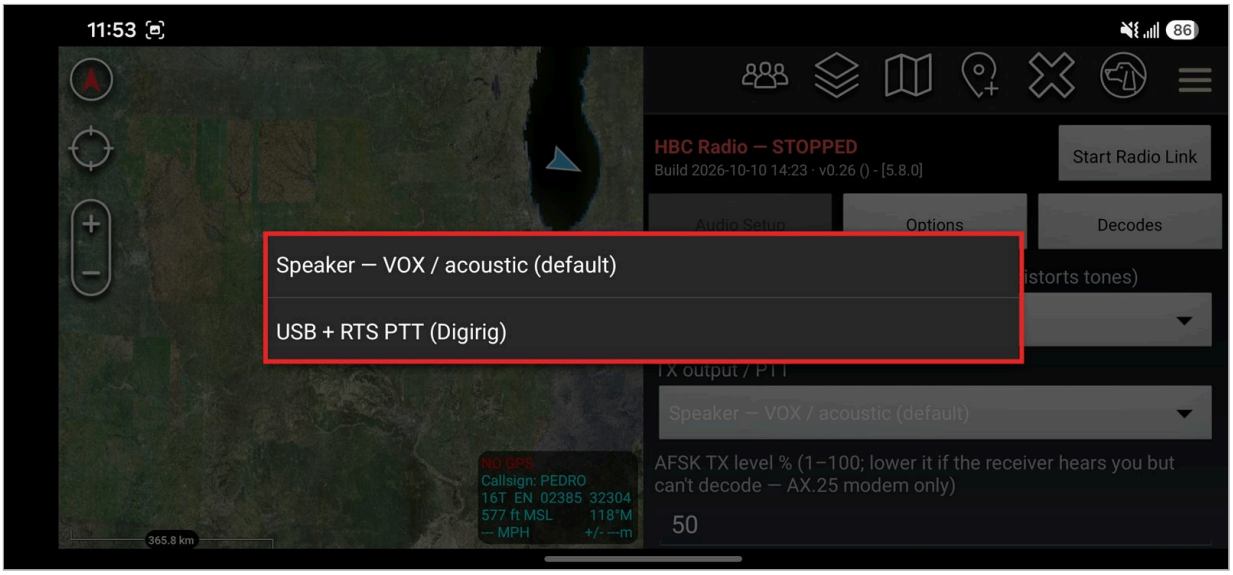


Figure 5 — TX output / PTT expanded (boxed): Speaker — VOX / acoustic coupling (default), or USB + RTS PTT for Digirig-class interfaces, where the plugin keys the radio electrically (sections 6.2 and 3.6).

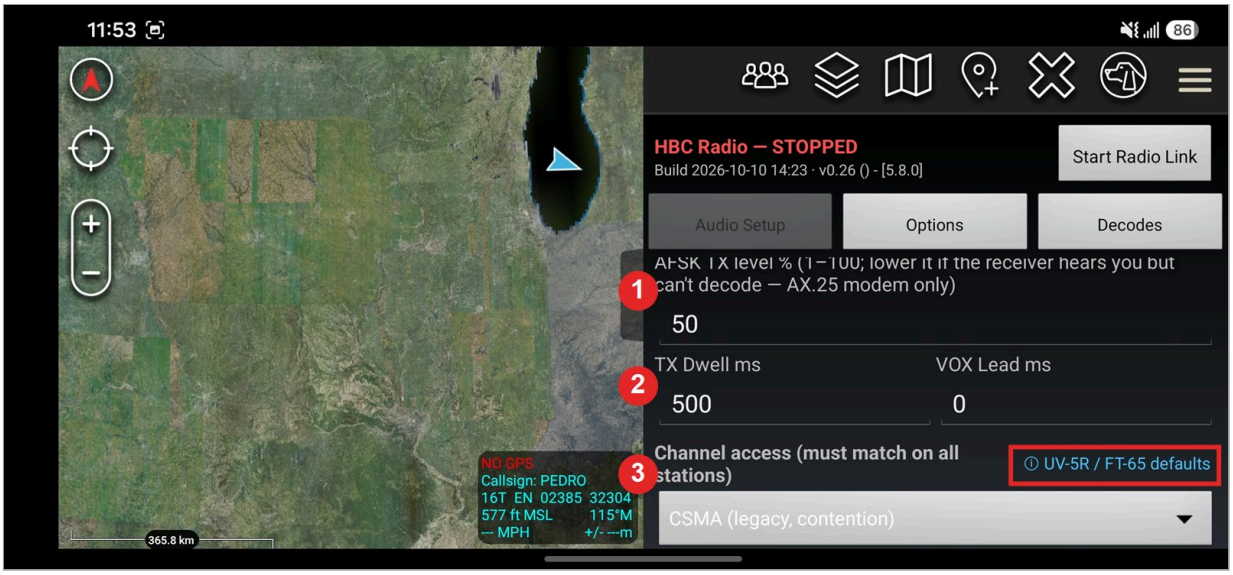


Figure 6 — Audio Setup, lower half. ① AFSK TX level % (AX.25 modem only); ② TX Dwell and VOX Lead; ③ Channel access selector with the ⓘ UV-5R / FT-65 defaults link (boxed) that opens the radio-profile dialog (section 3.6).



Figure 7 — Channel access expanded: CSMA (contention, section 3.5) or Ring (deterministic turns, section 3.6), with the Guard / Skip / Max-turn fields beneath (boxed).

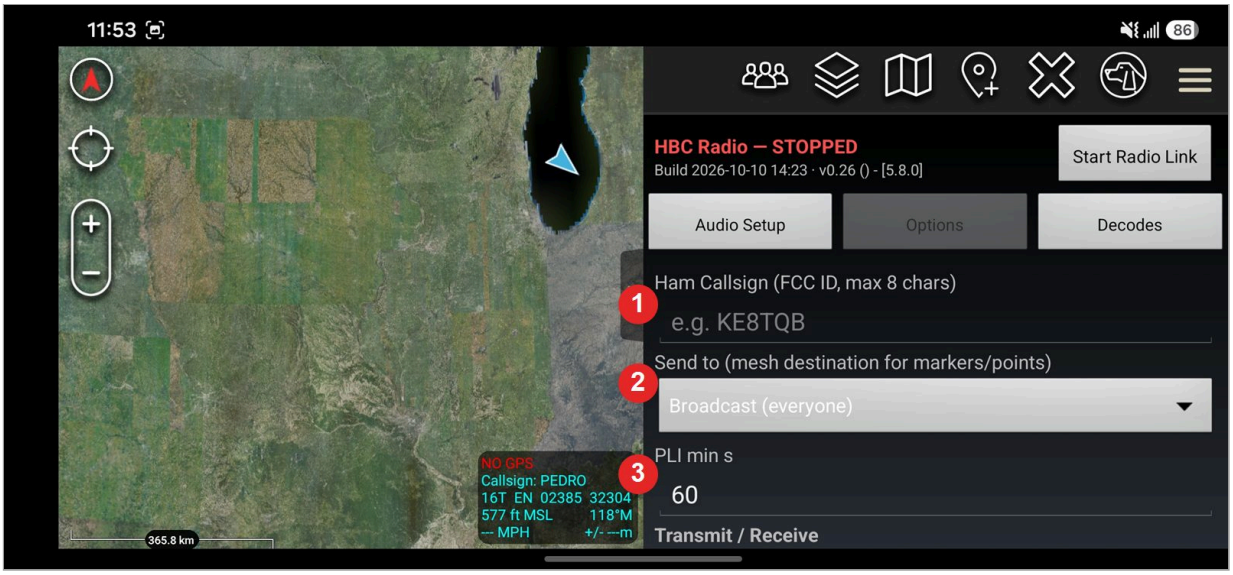


Figure 8 — Options tab, top. ① Ham callsign (required before starting); ② Send-to destination — Broadcast (everyone) or any station learned by the mesh (section 4.7); ③ PLI min s rate limit.

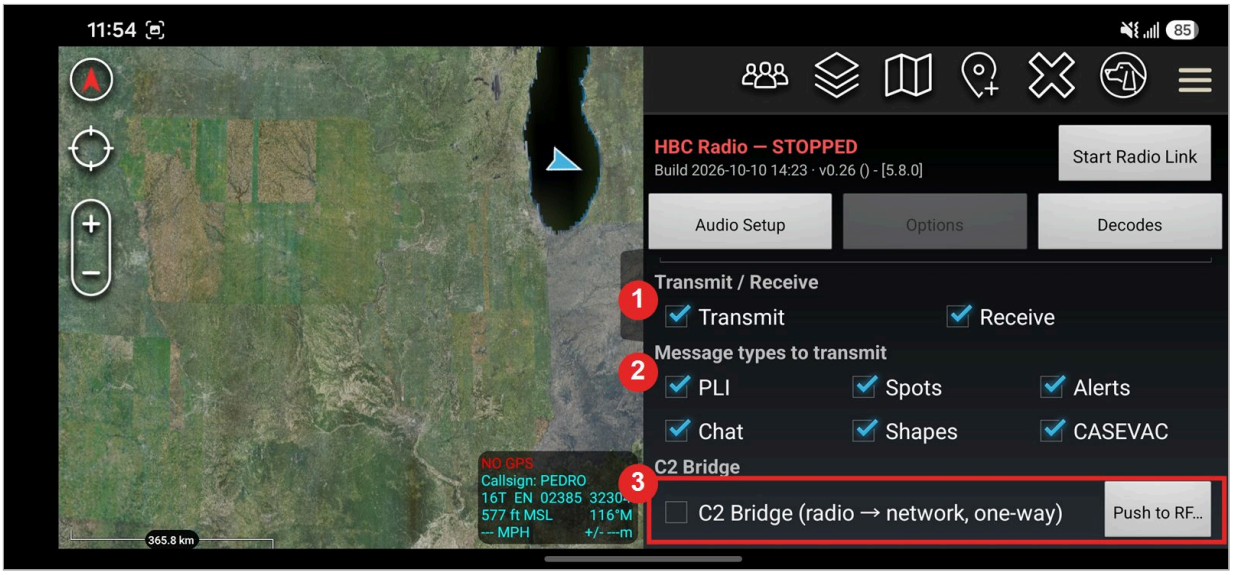


Figure 9 — Options tab, bottom. ① Transmit / Receive master switches; ② per-type transmit filters (the HBC modes of section 5.2); ③ the C2 Bridge checkbox with its Push to RF… button (boxed) — the one-way data diode of section 2.1.

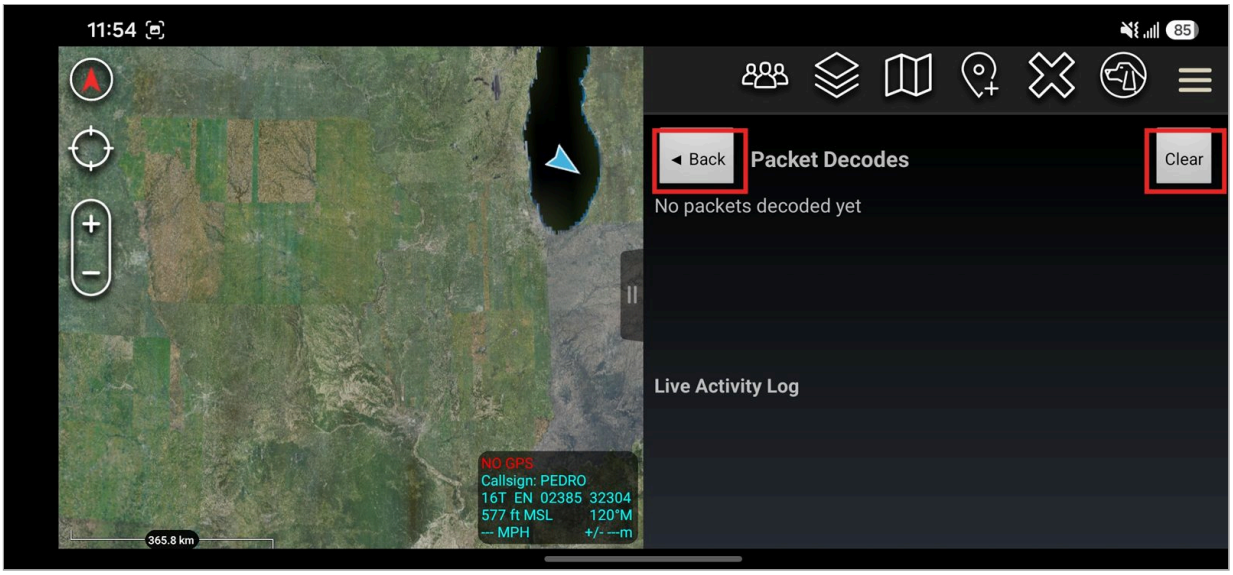


Figure 10 — The Decodes page takes over the whole pane: ◄ Back (boxed, left) returns to the last settings tab, Clear (boxed, right) resets the counter; the Packet Decodes list sits above the Live Activity Log (sections 6.4–6.5).

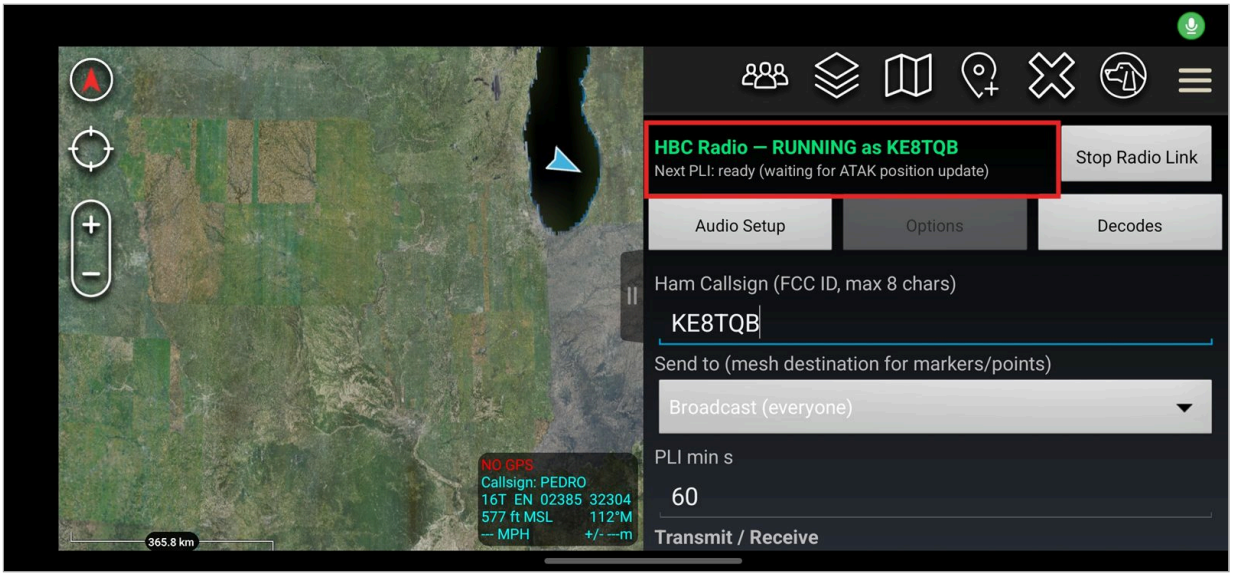


Figure 11 — Running: the header (boxed) turns green (`RUNNING as {callsign}`) with the live Next-PLI line beneath it, and the button flips to Stop Radio Link.

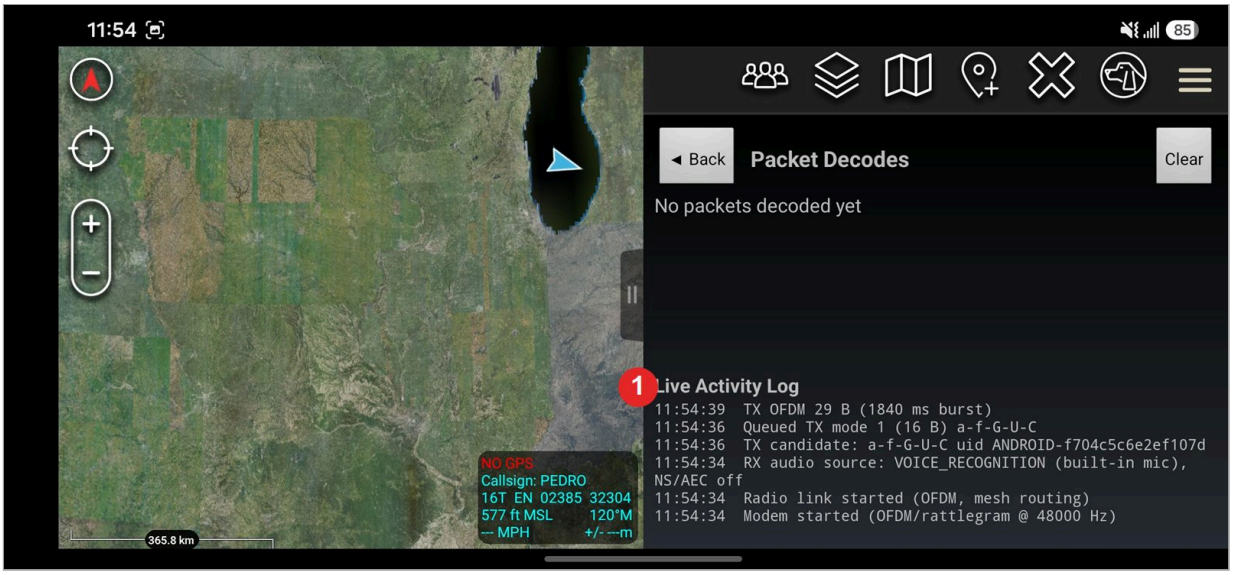


Figure 12 — The Live Activity Log (①) seconds into a session: modem start, RX audio source, then a position report walking the TX pipeline (`TX candidate → Queued TX → TX OFDM … ms burst`) — the exact lines cataloged in section 6.4.

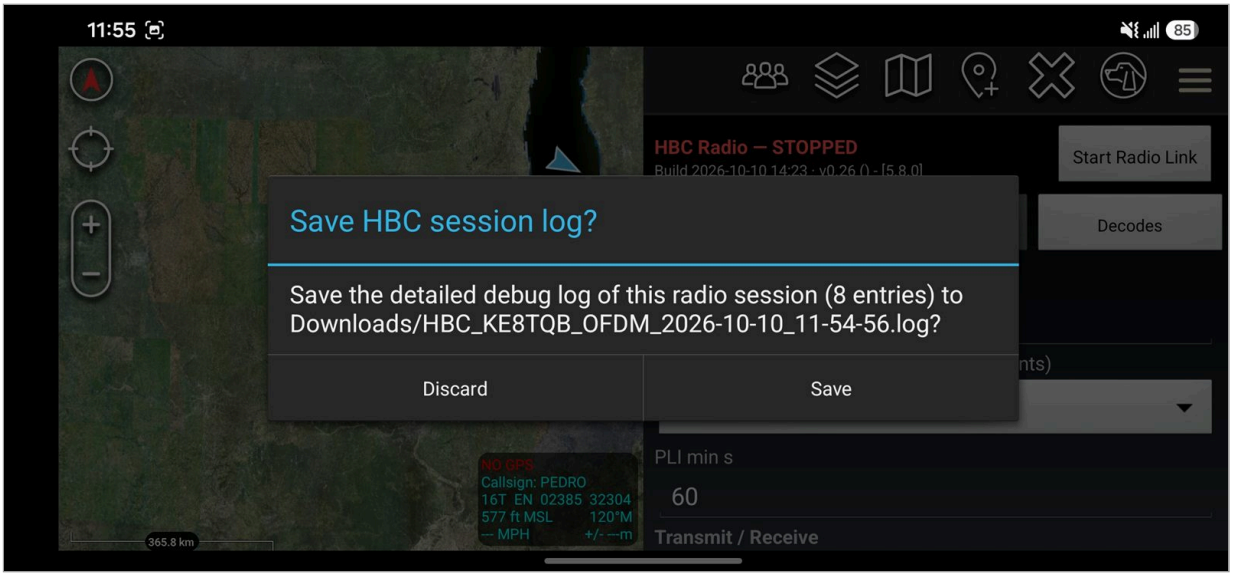


Figure 13 — Stopping the link offers the session's detailed debug log for saving to Downloads; the log's header line records the build stamp and every active setting (section 6.4).

6.2 Every Control and What It Does

Grouped by where each control lives in the pane:

Control	Type	What it does
Audio Setup / Options /	Tab buttons	Switch screens; the active tab's button is grayed out. **Decodes** opens as a full-panel page (header and tab bar hidden) with its own ◀ Back

Decodes		button returning to the last settings tab.
Status line	Text (header, always visible)	`HBC Radio — RUNNING as {callsign}` in **green** while the modem is on; `HBC Radio — STOPPED` in **red** otherwise.
Next PLI / Build line	Live text (1 s updates)	While running: countdown until the next position report may transmit (`in m:ss`, `ready (waiting for ATAK position update)`, or `disabled`). While stopped: `Build {timestamp} · v{version}` — use it to verify which binary is actually installed.
Ham Callsign	Text field	Your FCC callsign, max 8 characters (A–Z, 0–9, hyphen). Used as this station's mesh address and legal station ID. Required before starting.
Send to	Dropdown	Destination for markers/points you create: `Broadcast (everyone)`, or any heard station for a routed, acknowledged Direct (section 4.5). Chat DMs ignore this and always route to their recipient automatically.
TX Dwell ms	Number field	AFSK only: length of the HDLC flag preamble sent before data so the receiving modem can sync. Default 500. Increase if receivers miss the start of packets.
VOX Lead ms	Number field	Warm-up audio played before data so a VOX-keyed radio has time to key up. Default 0 (use 0 with a hardwired PTT interface). Increase if the first part of transmissions is cut off.
PLI min s	Number field	Minimum seconds between your own position reports (rate limit). Default 60. Drives the "Next PLI" countdown.
Modem	Dropdown	Which of the four waveforms to use (section 3). Listed OFDM (Rattlegram), Mercury, AX.25/APRS, PSK31 (very slow — PLI ≈15 s on air); **OFDM is the default**. Must match on every station.
TX audio stream	Dropdown	Which Android audio channel carries the transmit tones (Figure 4). Default Alarm; try the others only if your device distorts the tones.
TX output / PTT	Dropdown	Where transmit audio goes and how the radio is keyed. **Speaker — VOX / acoustic** (default): audio plays from the phone speaker into the radio's mic and the radio's VOX (or a finger on PTT) keys the transmitter; TX audio is explicitly pinned to the speaker even when a USB interface is attached. **USB + RTS PTT (Digirig)**: TX audio is routed to the USB audio interface on the MEDIA stream (set Media volume to max) and the plugin keys the radio electrically 60 ms before each burst: CP210x serial RTS on the Digirig Mobile, CM108 GPIO on the Digirig Lite — no VOX attack or hang, so Guard 300–500 ms and VOX Lead 0 apply (ⓘ dialog). Requires a one-time Android USB permission.
Channel access	Dropdown	CSMA (contention, section 3.5) or Ring (deterministic turns, section 3.6). Must match on every station.

ⓘ UV-5R / FT-65 defaults	Help link	Opens a dialog with the recommended Ring/VOX values for the reference radios, why their VOX hang cannot be shortened, the Guard calibration procedure, and hardwired-PTT alternatives (section 3.6).
Guard ms / Skip ms / Max turn ms	Number fields	Ring MAC timing (section 3.6): Guard covers the radios' VOX hang (default 1500), Skip covers VOX attack before skipping a silent station (default 1200), Max turn caps one batch (0 = automatic per modem). Same values on every station. A larger Max turn also fits more frames into each batch (faster queue drain at the cost of longer turns); PSK31 always sends exactly one frame per turn regardless.
C2 Bridge (radio → network, one-way)	Checkbox (Options tab)	Activates the one-way data diode of section 2.1: decoded radio traffic is re-published onto the ATAK network, while network-origin events are blocked from automatic radio transmission and stashed for Push to RF. Takes effect immediately; the header shows · **BRIDGE** while on.
Push to RF…	Button (Options tab)	Opens a multi-select list of recently blocked network-origin items (newest 20, kept 10 minutes). Confirming transmits the selected items over radio once, through the normal HBC encode/queue path — the deliberate operator exception to the diode.
Transmit / Receive	Checkboxes	Master switches. Unchecking Transmit makes the station receive-only; unchecking Receive ignores all incoming audio.
PLI, Spots, Alerts, Chat, Shapes, CASEVAC	Checkboxes	Per-type transmit filters, matching the HBC modes of section 5.2. An unchecked type is never transmitted (receiving is unaffected).
Start / Stop Radio Link	Button (header, always visible)	Validates and saves the settings, then starts (or stops) the selected modem and the mesh layer. The label flips to match the current state.
Activity Log	Scrolling text	Timestamped diagnostic feed of everything the plugin does (section 6.4). Newest entries on top.
Clear (Decodes tab)	Button	Empties the Decodes log and resets its packet counter.

6.3 Toolbar Icons

Icon	Tap	Notes
HBC Radio (radio-tower glyph)	Opens the plugin pane	The plugin's single entry point in ATAK's Tools menu (Figure 1). The Tools grid tints icons white, so the entry uses a transparent-background tower silhouette; the colored badge appears as the app icon in TAK Package Mgmt and the plugin manager.

> **NOTE:** there is no separate radio-status toolbar icon — the radio link is controlled entirely from the pane's Start/Stop button, and the link state is shown by the colored header status line (red STOPPED / green RUNNING).

6.4 Reading the Activity Log

Every line starts with a `HH:mm:ss` timestamp; the newest line is at the top. The tables below list the messages verbatim (`{braces}` mark values filled in at runtime) and what each one means.

Radio link and modem:

Message	Meaning
`Radio link started ({modem}, mesh routing)`	Start succeeded; the named modem and the mesh layer are running.
`Radio link stopped`	The modem was stopped (button, icon tap, or plugin shutdown).
`Start failed: {error}`	The modem could not start (usually a microphone/audio problem). The link is left stopped.
`Modem started ... / RX audio source: {source}`	Which microphone source the receiver opened (VOICE_RECOGNITION, UNPROCESSED, or MIC; "via USB" when a Digirig-style interface is attached).
`RX disabled: microphone unavailable`	ATAK lacks the RECORD_AUDIO permission or another app holds the mic; receiving is off.
`TX {details} ({n} ms burst)`	A transmission finished playing; `n` is its on-air duration.
`CSMA: channel busy — deferring TX`	Someone else is on the air; the plugin is politely waiting (section 3.5).
`CSMA: channel busy > {n} s — transmitting anyway`	The channel never cleared within the modem's patience window (8 s AFSK/OFDM, 15 s Mercury, 90 s PSK31), so the frame was sent to avoid starving traffic.
`OFDM sync: {call} ... / Mercury sync acquired`	The demodulator locked onto an incoming signal (decode result follows).
`TX batch: {n} frames ({t} ms burst)`	Several queued frames went out as one continuous keying (one VOX cycle for the whole batch).
`PTT: USB RTS ready ({name}) / PTT: USB GPIO ready ({name})`	The keyer is open — CP210x serial RTS (Digirig Mobile) or CM108 GPIO (Digirig Lite); every transmission keys the radio electrically.
`PTT: no Digirig PTT device found (CP210x serial or CM108 GPIO) — using VOX/manual keying`	USB + RTS PTT is selected but no keyer hardware is attached; audio still plays, but the radio must be keyed by VOX or by hand.

`PTT: TX audio uses the Media volume on USB — set Media volume to max`	Reminder on engage: over USB the tones ride the MEDIA stream; a low Media volume makes TX into the radio faint.
`PTT: requesting USB access — approve the dialog (RTS engages automatically)`	Android is showing its “Allow ATAK to access the USB device?” dialog; after Allow, RTS PTT engages by itself (`PTT: USB permission granted` follows). Denying logs `PTT: USB permission denied`.
`PTT: RTS write failed ({code}) — reconnect the Digirig and Stop/Start`	The USB device stopped accepting the keying command (unplugged/re-enumerated); reconnect and restart the link.
`PTT: Digirig attached — engaging / PTT: Digirig detached — RTS PTT off (VOX/manual keying)`	Hot-plug while the link runs: attaching a Digirig engages PTT automatically; detaching falls back to plain audio. Every USB attach/detach also re-opens the receive path (a fresh `RX audio source: …` line follows) — no Stop/Start needed after a reconnect.

> **NOTE:** plugging in the Digirig may also pop Android’s generic “Choose an app for the USB device” chooser. ATAK is not listed there (it registers no USB-launch handler) and that chooser is irrelevant to the plugin — dismiss it. The dialog that matters is the *Allow ATAK to access the USB device* permission prompt the plugin raises when the radio link starts in USB + RTS PTT mode.

Channel access (Ring MAC, section 3.6):

Message	Meaning
`Ring: started ({n} stations, guard {g} ms, skip {s} ms)`	Ring mode is on with these parameters and this roster size.
`Ring: joined rotation as {call} ({n} stations)`	The listening (settle) period is over; this station now takes its turns.
`Ring: TX turn ({i}/{n}, {x} frames)`	It is this station’s turn and {x} queued frames were released as one batch.
`Ring: roster now {n} stations ({list})`	The rotation list changed at a cycle wrap (station joined or expired).
`Ring: {call} dormant (3 silent turns)`	That station has not transmitted for 3 cycles; it keeps costing only the Skip time.
`Ring: emergency frame queued — preempting at next idle window / Ring: emergency TX ({x} frames, preempt)`	A 911 alert jumped the turn order in an idle gap (the single allowed exception).

`Ring: TX queue {n} frames (~{t} s to drain)`	More than 6 frames are waiting for this station's turns (estimate from the measured rotation time; logged at most every 30 s). If it persists, slow down chat/marker sends — see section 3.6's ARQ-interaction note.
`Ring: TX turn ({i}/{n}, {x} frames, forced — channel busy all turn)`	The busy interlock blocked this station's whole turn (open squelch, hot Digirig RX line, constant interference), so the batch transmitted at the deadline anyway — the Ring equivalent of CSMA's give-up override. Check radio RX volume/squelch if this repeats.
`TX ISSUE: {n} frames stuck in the TX queue — …`	Watchdog: more than 5 frames are waiting anywhere in the TX path and not draining; the channel reads permanently busy or the PTT/audio path failed. Logged with a toast at most every 30 s; Stop/Start recovers if it persists.

C2 Bridge (section 2.1):

Message	Meaning
`Bridge: {summary} -> network`	A decoded radio event was re-published onto the ATAK network outputs (TAK server / mesh SA).
`Bridge: blocked {type} {uid} (network-origin) — use Push to RF`	The diode stopped a network-origin event from auto-transmitting over radio; it is available in the Push to RF list for 10 minutes.
`Bridge: manual push mode {n} ({x} B) {type} -> RF`	The operator selected this item in Push to RF; it was encoded and queued for radio exactly like native traffic.
`Bridge: push skip: {reason}`	A selected item has no HBC representation (unsupported type/characters) and was not transmitted.

Transmit decisions:

Message	Meaning
`TX candidate: {type} uid {uid}`	ATAK asked to send this event; the plugin is deciding whether to transmit it.
`Queued TX mode {n} ({x} B) {type}`	The event was encoded as HBC mode `n` (`x` bytes) and handed to the mesh for transmission.
`Queued TX mode 0 ack ({type}) -> {call} tag 0x{tag}`	A chat delivered/read receipt is being sent back to a DM sender (section 5.4).
`TX skip: type {type} disabled in settings`	Dropped: that message type's checkbox is off.
`TX skip: PLI rate limit`	Dropped: a position report arrived before "PLI min s" elapsed (see the Next PLI countdown).

`TX skip: duplicate within 10 s`	Dropped: ATAK re-sent the same event; the copy was suppressed to save airtime.
`TX skip: {reason}`	Dropped: the event has no HBC representation (e.g. text with unsupported characters, coordinates out of range).

Mesh routing:

Message	Meaning
`Mesh: route {call} via {call2} ({n} hops)`	Address-book update: {call} is now reachable by handing frames to {call2}. A trailing `[heard]` means it was learned passively just by hearing traffic (section 4.7). The Send-to list refreshes.
`Mesh: announce sent (seq {n})`	The 10-minute quiet-time keepalive announce went out.
`Mesh: direct to {call} via {call2} (seq {n})`	A routed Direct left this station; an ACK is now awaited.
`Mesh: direct to {call} via {call2} (seq {n}, unacked)`	A fire-and-forget Direct (chat delivered/read receipt): no retries, no failure verdict (section 4.9).
`Mesh: {call} acknowledged seq {n}`	Delivery confirmed — the destination's ACK arrived.
`Mesh: retry {i}/3 to {call} via {call2}`	No ACK yet; the Direct was re-sent (same sequence ID). This fires only after the previous copy actually aired — under Ring, at least ~1.25 rotations later.
`Mesh: direct to {call} FAILED (no ack after 3 retries)`	Delivery gave up. The station is likely out of range or off. Also issued when queued copies could not reach the air for 3 minutes (stuck queue).
`Mesh: no route to {call} — sending as broadcast`	No address-book entry for the destination, so the message went out unrouted; the station will still get it if in direct range.
`Mesh: forwarding {call} -> {call2} via {call3}`	This station is acting as a relay for someone else's Direct.
`Mesh: no route to {call} — direct dropped`	Asked to relay a Direct but no route to the destination is known.

Receive decisions:

Message	Meaning
`RX {call} {summary}`	A packet was decoded and injected into ATAK (the same event also appears in the Decodes tab, section 6.5).

`RX chat: DM for '{call}' — not this station, ignored`	A direct chat addressed to someone else was overheard and correctly not shown.
`RX ack for '{call}' — not this station, ignored`	Same, for a chat receipt.
`RX ack tag 0x{tag} — no matching sent DM, ignored`	A receipt arrived for a message this station has no record of sending (e.g. app restarted).
`RX decode produced invalid CoT ({summary})`	The packet decoded but ATAK rejected the rebuilt event — report this as a bug.

6.5 Reading the Decodes Log

The Decodes page (Figure 10) lists only *successfully decoded* packets — a clean record of what actually arrived. The header shows a running total (`{n} packet(s) decoded`; Clear resets it). Each entry has three lines:

```
[{MM-dd HH:mm:ss}]  {origin callsign}
  {message summary}
  {n} B payload
```

The summary line identifies the HBC mode and its key contents:

Summary format	How to read it
`Mode 1 PLI {call} '{name}' @ {lat},{lon}`	Position report from {call}; the marker {name} was placed/updated at those coordinates. `Spot` instead of `PLI` means a placed marker, not a self-position.
`Mode 2 Alert(ACTIVE) {name} @ {lat},{lon}`	A 911 emergency from {name} at that location. `(CANCEL)` means the emergency was called off.
`Mode 3 Chat {call}: {text}`	A chat message and its text (broadcast, room, or DM — DMs for other stations are never injected).
`Mode 0 Ack(DELIVERED) {call} -> {call2} tag 0x{tag}`	{call} confirms a DM from {call2} was delivered; `(READ)` means it was opened. Produces the chat checkmarks.
`Mode 4 Shape({k}) {call} '{name}'`	A drawing (0 = circle, otherwise polygon/polyline) named {name}.
`Mode 5 CASEVAC {call} '{name}'`	A medical-evacuation request titled {name}.
`Mode 6 Marker {type} {call} '{name}' @ {lat},{lon}`	A full marker with its exact CoT type and icon (section 5.2, Mode 6).

> **Rule of thumb:** the Activity Log answers “what is the plugin doing and why?” (including everything it decided *not* to send); the Decodes log answers “what have I actually received?”

7. Timing Parameters and Constants

Parameter	Value	Layer
Route entry TTL	30 min (refreshed by any heard frame)	Mesh
Duplicate cache TTL	5 min	Mesh
Cleanup interval	60 s	Mesh
Direct retry timer	Armed when the frame airs. CSMA: 5 s + 0–2 s jitter; Ring: max(8 s, 1.25 × measured rotation), cap 60 s. Max 3 retries; stuck-queue fail 180 s	Mesh
Quiet-time announce keepalive	10 min without own traffic	Mesh
CSMA busy threshold / hold	4× adaptive noise floor / 400 ms; busy persisting past the modem's re-baseline window (10 s; 90 s on PSK31) folds into the noise floor so a hot RX line cannot latch the channel busy	PHY
CSMA backoff / give-up	150–550 ms clear / 8 s (15 s Mercury, 90 s PSK31)	PHY
Ring Guard (VOX hang cover)	User setting, default 1500 ms (UV-5R / FT-65 profile)	MAC
Ring Skip (silent-station timeout)	User setting, default 1200 ms	MAC
Ring Max turn	Auto per modem: AFSK 6000 / OFDM 4400 / Mercury 6000 / PSK31 ~94000 ms, sized from the worst-case frame airtime (user-overridable)	MAC
Ring settle (join listening)	1 observed cycle, max 30 s	MAC
Ring batch caps	Derived from Max turn; at the automatic defaults: AFSK 4 / OFDM 2 / Mercury 1 / PSK31 1 frames per keying (also applies under CSMA)	MAC/PHY
PLI rate limit	User setting, default 60 s (Figure 8); Ring mode floors it at roster × 8 s (× 13 s Mercury, × 25 s PSK31)	Application
PSK31 frame airtime	≈0.4 s per byte + ~4 s preamble/overhead (PLI ≈ 16 s; 180-byte maximum frame ≈ 77 s)	PHY
RX duplicate window (frame hash)	30 s	Plugin

8. Compatibility and Constraints

- All stations on a channel must run plugin builds with the same mesh framing and select the same modem (Figure 3).
- The channel-access mode (section 3.5/3.6) is **behavioral only** — nothing on the air changes. For orderly rotation every station should select the same mode and the same Guard/Skip values; a CSMA station mixed into a Ring net simply contends as before and the Ring stations defer to its carrier (the net degrades toward polite CSMA, never below it).
- The C2 Bridge (section 2.1) is local behavior on the bridge phone only — one-way RF → network by design. Radio-side stations hear nothing new on the air except items the bridge operator deliberately pushes; network clients just receive ordinary CoT.
- The TX output / RTS PTT selector only changes how the local station keys its own radio — the waveforms are identical, so VOX-keyed and RTS-keyed stations mix freely on one channel (RTS stations may simply run tighter Guard timing).
- Mercury DATAC4's small frame limits routed Directs to ~29 bytes of HBC payload (Broadcasts ~39 B). Every standard message fits; unusually large Mode 6 markers may need Broadcast or a different modem.
- PSK31 accepts mesh frames up to 180 bytes but is extremely slow (section 3.4): a position report costs ~16 s of air. Prefer the Ring MAC whenever three or more stations share a PSK31 channel, and disable or maximize radio time-out timers (bursts can exceed 60 s).
- Callsigns: at most 8 ITA2 characters (A–Z, 0–9, hyphen).
- No encryption is used or permitted; never transmit sensitive data.

9. Glossary

Term	Meaning
ATAK	Android Team Awareness Kit — the mapping app this plugin extends
CoT	Cursor on Target — the XML message format ATAK uses for map events
EUD	End User Device — the phone/tablet running ATAK
PLI	Position Location Information — a "here I am" report
Modem	Modulator/demodulator — converts bytes to sound and back
Frame / packet	One complete chunk of bytes sent over the air
Payload	The useful data inside a packet, after the addressing header
Hop	One radio link in a relay chain (A→B is one hop; A→B→C is two)
Route / next hop	Address-book entry: which neighbor to hand a message to so it reaches a destination

ACK	Acknowledgment — a short “received OK” reply
ARQ	Automatic Repeat reQuest — send, await the ACK, re-send on timeout (section 4.9)
C2 Bridge / data diode	The one-way gateway (section 2.1): radio traffic flows to the network automatically; network traffic reaches the radio only by deliberate operator action
CSMA	Carrier Sense Multiple Access — listen before transmitting to avoid collisions
MAC	Medium Access Control — the rules deciding when a station may transmit (CSMA or Ring here)
Ring MAC / turn	The deterministic channel-access mode (section 3.6): stations rotate through the sorted roster; a turn is one station’s transmit window
PSK31 / Varicode	A 31.25-baud amateur keyboard mode and its variable-length text alphabet (ITU-R M.2034); HBC reuses the waveform to carry armored binary frames (section 3.4)
Guard time	Silence a Ring station waits after the channel clears, covering the previous radio’s VOX hang
VOX	Voice-operated transmit — the radio keys up automatically when it hears audio
CRC	Cyclic Redundancy Check — a checksum that detects corrupted data
ITA2	A 5-bit teletype alphabet from the telegraph era; smaller than normal 8-bit text
TTL	Time To Live — how long a record is kept before being erased
UID	Unique identifier string for a map object in ATAK

10. References

1. HBC Protocol reference implementation and specification — github.com/ke8tqb/HBC-Protocol
2. HBC ATAK Plugin source — github.com/ke8tqb/HBC-ATAK-Plugin
3. *Adaptation of Uncoordinated Distance-Vector Routing for Unencrypted Amateur Radio Networks* (routing protocol paper)
4. Mercury HF modem — mercury.hermes.radio; FreeDV/codec2 — github.com/drowe67/codec2
5. rattlegram / COFDMTV — github.com/aicodix/rattlegram
6. ITU-R Recommendation M.2034 — the PSK31 (31.25 Bd BPSK + Varicode) amateur standard carried by the PSK31 modem (section 3.4)

Appendix A — GitHub Repository File Structure

The plugin source lives at github.com/ke8tqb/HBC-ATAK-Plugin. The map below shows where everything described in this document is implemented.

```
HBC-ATAK-Plugin/
├── app/
│   ├── build.gradle                  Plugin version, ATAK version, NDK pin, signing config
│   ├── proguard-gradle.txt           Release-build shrinker rules
│   └── src/main/
│       ├── AndroidManifest.xml       Plugin discovery entry required by ATAK
│       ├── java/
│       │   ├── com/atakmap/android/hbc/
│       │   │   ├── HbcEncoder.java       Application layer: CoT XML → HBC bytes (Section 5)
│       │   │   ├── HbcDecoder.java       Application layer: HBC bytes → CoT XML
│       │   │   ├── Ita2.java             HBC's 5-bit text alphabet (Section 5.1)
│       │   │   ├── BitReader.java        Bit-level read helper for the codec
│       │   │   ├── BitWriter.java        Bit-level write helper for the codec
│       │   │   ├── MeshRouter.java       Mesh network layer: packets, routing,
│       │   │   │                         retries, ACKs (Section 4, 4.9)
│       │   │   ├── RingMac.java          Ring MAC turn rotation (Section 3.6)
│       │   │   ├── BridgePolicy.java     C2 Bridge data-diode rules + push stash (Section 2.1)
│       │   │   ├── PttKeyer.java         Hardware-PTT hook used by the modems
│       │   │   ├── CsmaSense.java        Listen-before-transmit logic (Section 3.5)
│       │   │   ├── AudioModem.java       AFSK1200 modem (Section 3.1)
│       │   │   ├── OfdmModem.java        OFDM COFDMTV modem (Section 3.2)
│       │   │   ├── OfdmNative.java       Java ⇔ C++ bridge for the OFDM library
│       │   │   ├── MercuryModem.java     Mercury HF modem (Section 3.3)
│       │   │   ├── MercuryNative.java    Java ⇔ C bridge for the FreeDV library
│       │   │   ├── Psk31.java            BPSK31 engine: Varicode, filters, AFC (Section 3.4)
│       │   │   ├── Psk31Armor.java       Text armor + CRC-16 + airtime estimator (Section 3.4)
│       │   │   ├── Psk31Modem.java       PSK31 modem (Section 3.4)
│       │   │   └── plugin/
│       │   │       ├── HbcPlugin.java    The plugin itself: settings pane, toolbar
│       │   │       │                     icons, TX/RX plumbing, ATAK integration
│       │   │       └── UsbPtt.java       Digirig RTS/GPIO PTT driver
│       │   └── sivantoledo/ax25/...      Bundled javAX25 library (AFSK/AX.25, GPL)
│       ├── cpp/
│       │   ├── CMakeLists.txt            Native build: libhbcofdm + libhbcmercury
│       │   ├── hbc-ofdm-jni.cpp          C++ glue for the OFDM modem
│       │   ├── *.hh                      Vendored aicodix OFDM/DSP headers
│       │   ├── hbc-mercury-jni.cpp       C glue for the FreeDV DATAC modem
│       │   ├── hbc-mercury-compat.c      Math shims for older Android versions
│       │   └── mercury/                  Vendored FreeDV/codec2 sources (from the
│       │                                 Mercury project; LGPL — see THIRD-PARTY.md)
│       └── res/
│           ├── layout/main_layout.xml    The settings/decodes pane (Figures 1–6)
│           ├── values/strings.xml        All UI text and dropdown options
│           └── drawable/                 Icons (ic_tools Tools-grid glyph, ic_launcher badge)
├── codec-test/
│   ├── HbcCodecTest.java             Byte-exact tests for the HBC codec (runs on a PC)
```

```
│   ├── Mode6Test.java                Byte-exact tests for Mode 6 extended markers
│   ├── MeshRouterTest.java           Byte-exact + behavior tests for the mesh layer
│   ├── RingMacTest.java              Ring MAC turn rules under a scripted clock
│   ├── Psk31LinkTest.java            PSK31 armor/modem link tests (audio loopback)
│   └── BridgePolicyTest.java         C2 Bridge diode/forward/stash rules
├── docs/
│   ├── HBC_ICD.html                  This document (editable source)
│   ├── HBC_ICD.pdf                   This document (rendered)
│   └── images/                       Figures 1–13 (annotated screenshots)
├── gradle/, gradlew, gradlew.bat     Gradle build system wrapper
├── settings.gradle, build.gradle     Project-level build configuration
├── .takdev/                          TAK SDK build-plugin support files
├── CHANGELOG.md                      Version-by-version history of changes
├── THIRD-PARTY.md                    Licenses of all bundled third-party code
└── README.md                         Project introduction and build instructions
```

> **Where to look first:** the wire formats of Section 4 are all in `MeshRouter.java`; the message encodings of Section 5 are in `HbcEncoder.java`/`HbcDecoder.java`; anything you can tap in the app is in `HbcPlugin.java` plus `res/layout/main_layout.xml`. The `codec-test/` programs run on an ordinary computer (no phone needed) and prove the byte formats match this document.